

PRACTICAL WEEK 11

Aim:

To develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join) and demonstrate race conditions when multiple threads update a shared variable concurrently.

Program:

```
sindhuj@SINDHUJA: ~/pract × + v
GNU nano 7.2 practical11_task1.c *
#include <stdio.h>
#include <pthread.h>

int counter = 0;

void *increment(void *arg)
{
    for (int i = 0; i < 100000; i++)
    {
        counter++;
    }
    return NULL;
}

int main()
{
    pthread_t t1, t2;

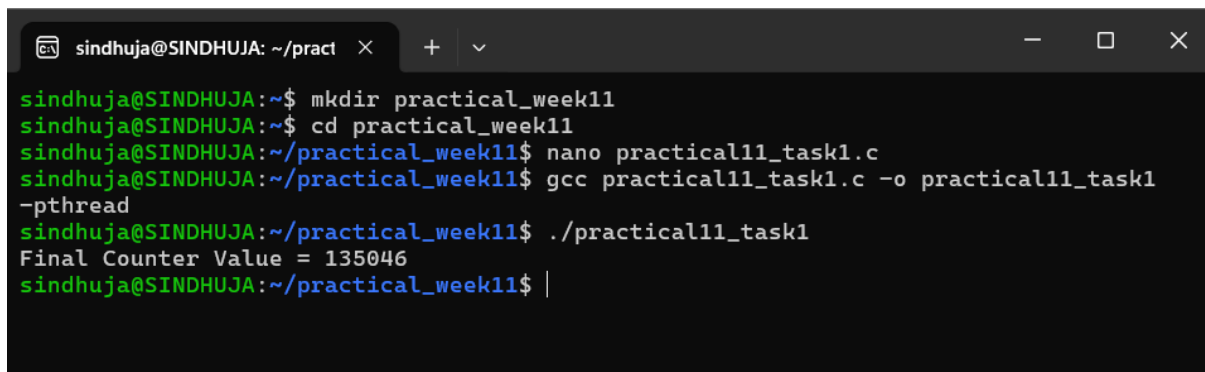
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final Counter Value = %d\n", counter);

    return 0;
}
```

Output:

A terminal window with a dark background and light green text. The window title bar shows 'sindhuja@SINDHUJA: ~/pract' and standard window controls. The terminal output shows the user creating a directory, navigating to it, creating a file with nano, compiling it with gcc, and running it. The output of the program is 'Final Counter Value = 135046'.

```
sindhuja@SINDHUJA:~$ mkdir practical_week11
sindhuja@SINDHUJA:~$ cd practical_week11
sindhuja@SINDHUJA:~/practical_week11$ nano practical11_task1.c
sindhuja@SINDHUJA:~/practical_week11$ gcc practical11_task1.c -o practical11_task1 -pthread
sindhuja@SINDHUJA:~/practical_week11$ ./practical11_task1
Final Counter Value = 135046
sindhuja@SINDHUJA:~/practical_week11$ |
```

Observation:

1. Two threads were created using `pthread_create()`.
2. Both threads updated the same shared variable.
3. `pthread_join()` waited for both threads to finish.
4. The final counter value changed across executions.
5. The variation confirmed the presence of a race condition.

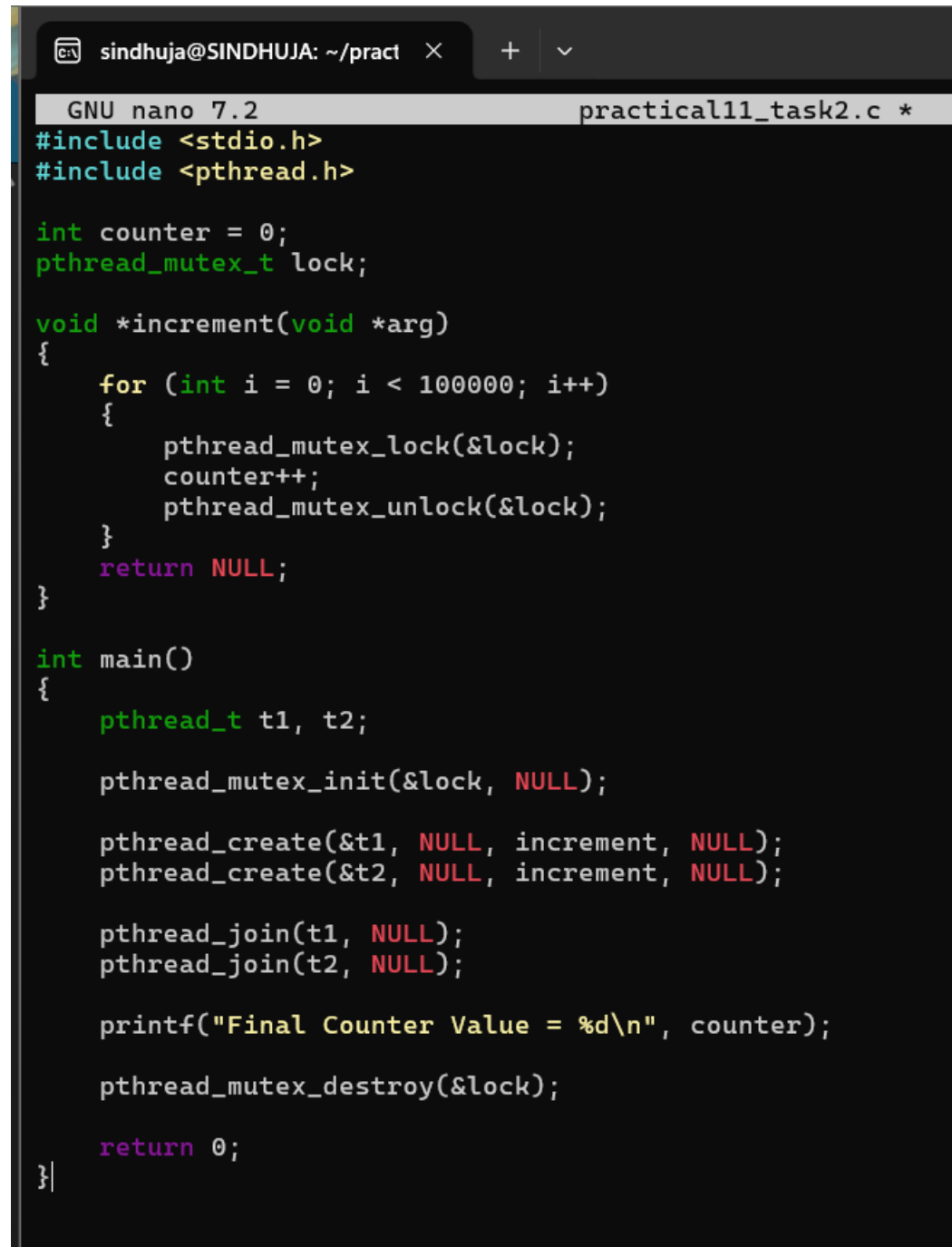
Result:

Successfully created a multithreaded counter application using POSIX threads and demonstrated race conditions caused by concurrent updates to a shared variable.

Aim:

To synchronize multiple threads using mutex locks (`pthread_mutex_lock` and `pthread_mutex_unlock`) and compare the synchronized output with the race-condition version.

Program:



```
sindhuj@SINDHUJA: ~/pract x + v
GNU nano 7.2 practical11_task2.c *
#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock;

void *increment(void *arg)
{
    for (int i = 0; i < 100000; i++)
    {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&lock, NULL);

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final Counter Value = %d\n", counter);

    pthread_mutex_destroy(&lock);

    return 0;
}
```

```

sindhuja@SINDHUJA:~/practical_week11$ nano practical11_task2.c
sindhuja@SINDHUJA:~/practical_week11$ gcc practical11_task2.c -o practical11_task2 -pthread
sindhuja@SINDHUJA:~/practical_week11$ ./practical11_task2
Final Counter Value = 200000
sindhuja@SINDHUJA:~/practical_week11$ echo "Race Condition Version:"
Race Condition Version:
sindhuja@SINDHUJA:~/practical_week11$ ./practical11_task1
Final Counter Value = 100782
sindhuja@SINDHUJA:~/practical_week11$ echo "Mutex Version:"
./practical11_task2
Mutex Version:
Final Counter Value = 200000
sindhuja@SINDHUJA:~/practical_week11$ |

```

Observation:

1. A mutex lock synchronized access to the shared variable.
2. Only one thread updated the counter at a time.
3. The final counter value remained 200000 in every execution.
4. Synchronization removed the race condition.
5. The mutex version produced reliable and consistent results.

Result:

Successfully synchronized multiple threads using `pthread_mutex_lock()` and `pthread_mutex_unlock()`, eliminated race conditions, and verified consistent counter values through synchronization.